Technical Report — CCA 3 Practical Assignment (Units 3, 4 and 5)

Student Name	Shivraj Maske
Roll number	1272250999
GitHub Link	https://github.com/shivrajmaske77/ComputerVisionProject
Course	Computer Vision – CCA 3

1. Objective

The objective is to implement and compare computer-vision techniques from three units using the replacement image and video files supplied for this project:

- **Unit 3** – image restoration (noise removal), morphological processing and segmentation,
- **Unit 4** – feature extraction (HOG, SIFT) and image matching,
- **Unit 5** – object recognition (Viola–Jones, YOLO) and motion estimation (frame differencing, GMM, Lucas–Kanade).

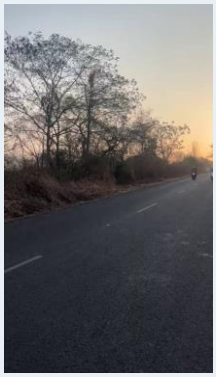
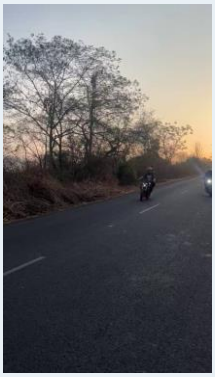
All code is in Python using OpenCV, NumPy, Matplotlib, scikit-image and Ultralytics YOLO.

2. Input data

File	Description	Source/device	Resolution / duration
`data/photo1.jpeg`	Restaurant/café interior with tables, chairs and counter	Not provided	963 × 1280 px original; resized to 800 × 1063 px for processing
`data/photo2.jpeg`	Interior of the same venue, with people visible	Not provided	720 × 1280 px original; processed at 720 × 1280 px
`data/objects.jpeg`	Personal-care and everyday items laid out on a dark surface; watershed input	Not provided	4865 × 3474 px original; resized to 800 px wide for processing
`data/photo5.jpeg`	Image containing several people, used for Unit 5 recognition	Not provided	612 × 380 px original; processed at its available size
`data/video.mp4`	Portrait-format video used for motion estimation	Not provided	4.95 s, approximately 60 fps, 297 frames; processing frames resized to 480 × 270 px

The images and video were read from the project data folder. Photos used by Units 3 and 4 are resized to 800 pixels wide; Unit 5 video frames are resized to 480 × 270 pixels to reduce computation. The recognition photo is already smaller than the 800-pixel target.

	
Figure 1 – Supplied restaurant/café interior	*Figure 2 – Supplied view of the same venue with people*

		
Figure 3 – Video frame at 25%	*Video frame at 50%*	*Video frame at 75%*

3. Source code

Notebook	Tasks
`Unit3_Restoration_Segmentation.ipynb`	Task 1 – restoration and segmentation
`Unit4_Feature_Extraction.ipynb`	Task 2 – HOG, SIFT, matching
`Unit5_Recognition_Motion.ipynb`	Task 3 – Viola–Jones and YOLO; Task 4 – motion estimation

Setup and run instructions are in `README.md`. Every notebook can be run with *Restart & Run All*.

Requirement: OpenCV 4.x (`opencv-python>=4.8,<5`). OpenCV 5 moved the Haar cascade face detector out of the main package, so `cv2.CascadeClassifier` is missing there and the Viola–Jones cell fails.

4. Task 1 — Image restoration and segmentation (Unit 3)

4.1 Adding noise

Gaussian noise (mean 0, standard deviation 25) was added to every pixel, and salt-and-pepper noise was added by setting 5% of pixels to black and 5% to white.

		
Figure 4 – Original	*Gaussian noise (σ = 25)*	*Salt-and-pepper noise (10%)*

4.2 Restoration with Gaussian and median filters (5 × 5)

The Gaussian filter replaces each pixel with a weighted average of its neighbours. The median filter replaces it with the middle value of its sorted neighbours, which throws away extreme values.

	Gaussian filter	**Median filter**
Gaussian noise		
Salt-and-pepper noise		

Figure 5 – Restored images.

4.3 Quality measurement (PSNR and SSIM)

PSNR measures pixel error in decibels (higher is better); SSIM measures structural similarity from 0 to 1 (1 = identical).

Image	**PSNR (dB)**	**SSIM**
Gaussian noisy (no filter)	20.70	0.319
Gaussian noise → Gaussian filter	27.49	0.704
Gaussian noise → Median filter	26.55	0.671
S&P noisy (no filter)	14.69	0.200

Image	PSNR (dB)	SSIM
S&P noise → Gaussian filter	23.44	0.494
S&P noise → Median filter	28.35	0.852

Observation: Both filters improved PSNR and SSIM over the noisy inputs. For Gaussian noise, the Gaussian filter scored higher (27.49 dB, SSIM 0.704) than the median filter (26.55 dB, 0.671), consistent with averaging reducing distributed random noise. For salt-and-pepper noise, the median filter performed better (28.35 dB, SSIM 0.852) than the Gaussian filter (23.44 dB, 0.494), because median filtering removes isolated extreme pixels without spreading them as much.

4.4 Thresholding

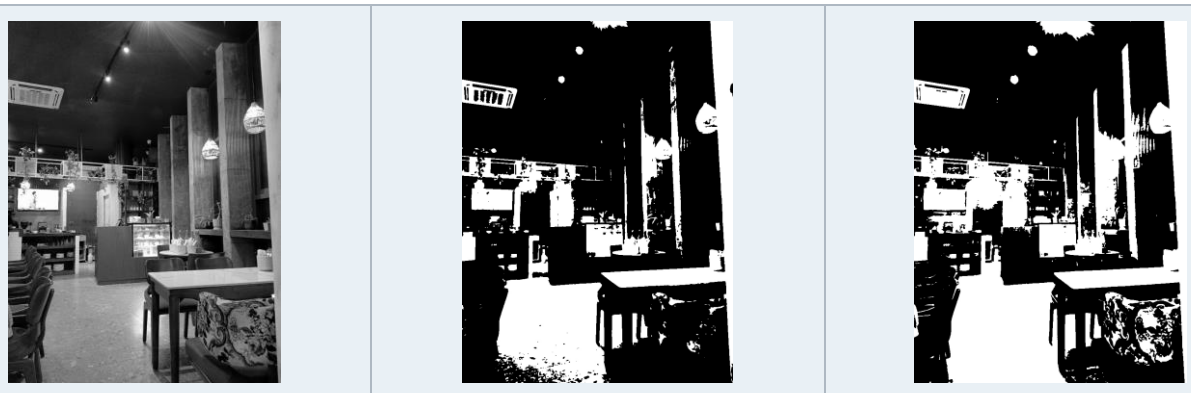
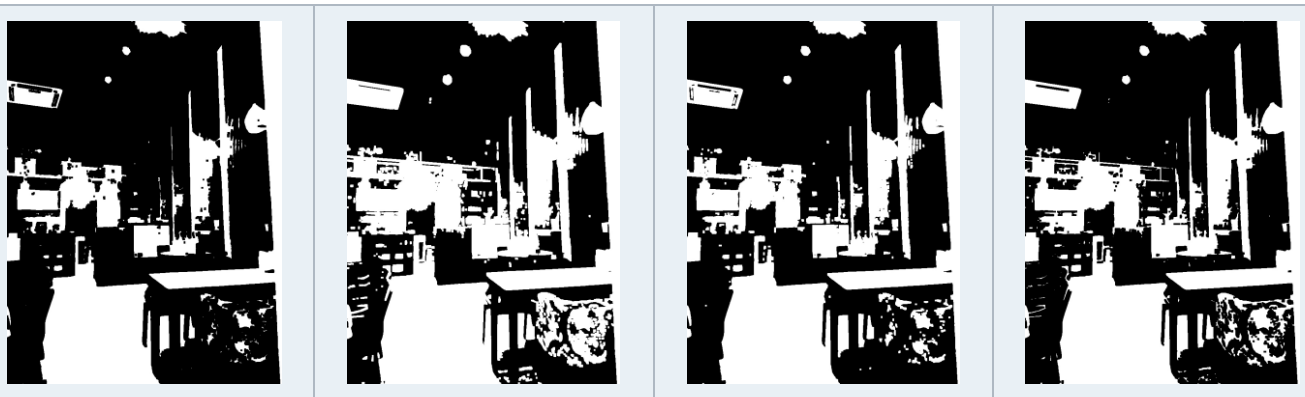


Figure 6 – Grayscale	*Fixed threshold (127)*	*Otsu threshold (100)*

Observation: Otsu selected a threshold of 100, 27 grey levels below the fixed value of 127. Otsu adapts the threshold to the grayscale histogram; the fixed threshold remains a useful baseline but does not adapt to this interior image's lighting and shadows.

4.5 Morphological operations (5 × 5 square kernel)



Erosion	*Dilation*	*Opening*	*Closing*

Figure 7 – Morphological results on the Otsu binary image.

Operation	White pixels (%)
Binary (Otsu)	36.6
Erosion	30.3
Dilation	43.7
Opening	35.0

Operation	White pixels (%)
Closing	37.6

Observation: Erosion reduced the white area from 36.6% to 30.3%, while dilation increased it to 43.7%. Opening resulted in 35.0% white pixels and closing in 37.6%, both closer to the original mask area than erosion or dilation alone.

4.6 Canny edge detection



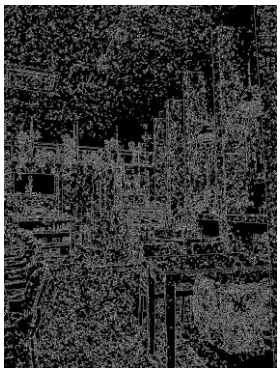
No blur (100, 200)



Blurred, low thresholds (30, 100)



Blurred, high thresholds (100, 200)



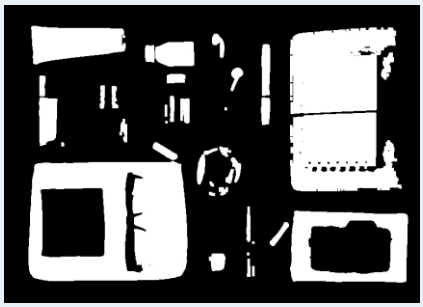
On the Gaussian-noisy image (100, 200)

Figure 8 – Canny edge maps.

Setting	Edge pixels (%)
No blur (100, 200)	6.52
Blurred, low (30, 100)	6.77
Blurred, high (100, 200)	3.96
Noisy image (100, 200)	19.36

Observation: Blurring and using thresholds (100, 200) produced the fewest edges (3.96%). The Gaussian-noisy image produced 19.36% edge pixels, substantially more than the blurred result, demonstrating how noise can generate spurious edges. The low-threshold blurred result (6.77%) was slightly higher than the unblurred result (6.52%), while the high-threshold blurred result suppressed weaker edges.

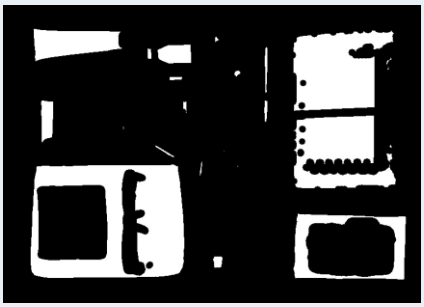
4.7 Watershed segmentation



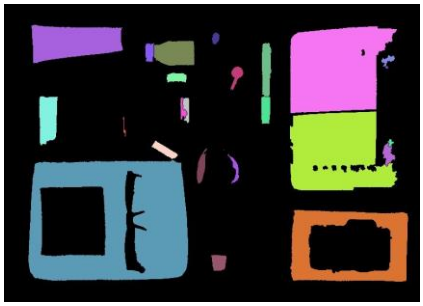
Binary mask



Distance transform



Sure foreground (markers)



Watershed regions



Boundaries (red)

Figure 9 – Marker-based watershed on the supplied flat-lay image.

Result: The supplied flat-lay has multiple separate items on a textured background; no reliable ground-truth object count was provided.

Method	Segmentation regions
Plain binary mask, all blobs	41
Watershed, `FG_RATIO` = 0.1	25

The supplied flat-lay contains many distinct items on a textured dark surface rather than a clean set of touching objects on a plain background. Thresholding and watershed returned 41 connected foreground blobs and 25 watershed regions at `FG_RATIO = 0.1`. These counts are segmentation regions, not a verified count of physical items: shadows, texture, and adjacent/overlapping items can fragment or merge regions. The algorithm demonstrates marker-based segmentation, but the result should not be interpreted as an accurate object count for this input.

4.8 Comparison — Task 1

Job	Techniques compared	Best on supplied images	Reason
Gaussian noise	Gaussian vs median filter	Gaussian filter	27.49 dB / SSIM 0.704 against 26.55 dB / 0.671; averaging reduces distributed random noise
Salt-and-pepper noise	Gaussian vs median filter	Median filter	28.35 dB / SSIM 0.852 against 23.44 dB / 0.494; median filtering removes isolated extreme values
Binary mask	Fixed vs Otsu	Otsu	Selected threshold 100 from the image histogram, compared with fixed threshold 127
Morphology	Erosion, dilation, opening, closing	Opening and closing stayed closest to the original mask area	Otsu mask 36.6%; opening 35.0%; closing 37.6%
Edges	Canny settings	Blurred, (100, 200)	3.96% edge pixels against 6.77% (blurred, low thresholds) and 19.36% (noisy image)

Job	Techniques compared	Best on supplied images	Reason
Flat-lay segmentation	Plain mask vs watershed	Watershed produced more separated regions	41 connected blobs in the plain mask and 25 watershed regions at `FG_RATIO = 0.1`; these are not verified object counts

5. Task 2 — Feature extraction and matching (Unit 4)

5.1 HOG features

HOG settings: 9 orientation bins, 16 × 16 pixel cells, and 2 × 2 cell blocks. Feature vector lengths were **114,660** for photo 1 (1063 × 800 px) and **125,136** for photo 2 (1280 × 720 px). The vector size differs because the source images have different aspect ratios and dimensions.

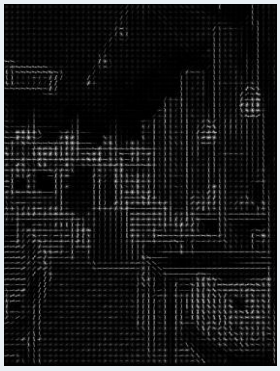
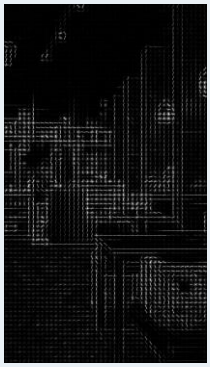


Figure 10 – HOG of photo 1



HOG of photo 2

5.2 SIFT keypoints

Photo 1: 4,053 keypoints; photo 2: 3,553 keypoints. Each SIFT descriptor has 128 values. Circles show keypoint scale and the line shows orientation.



Figure 11 – SIFT keypoints, photo 1



SIFT keypoints, photo 2

5.3 Matching and the rotation/resize experiment

Matching: brute-force (Euclidean distance) with k = 2, Lowe's ratio test (0.75), and RANSAC homography to count correct matches (inliers).

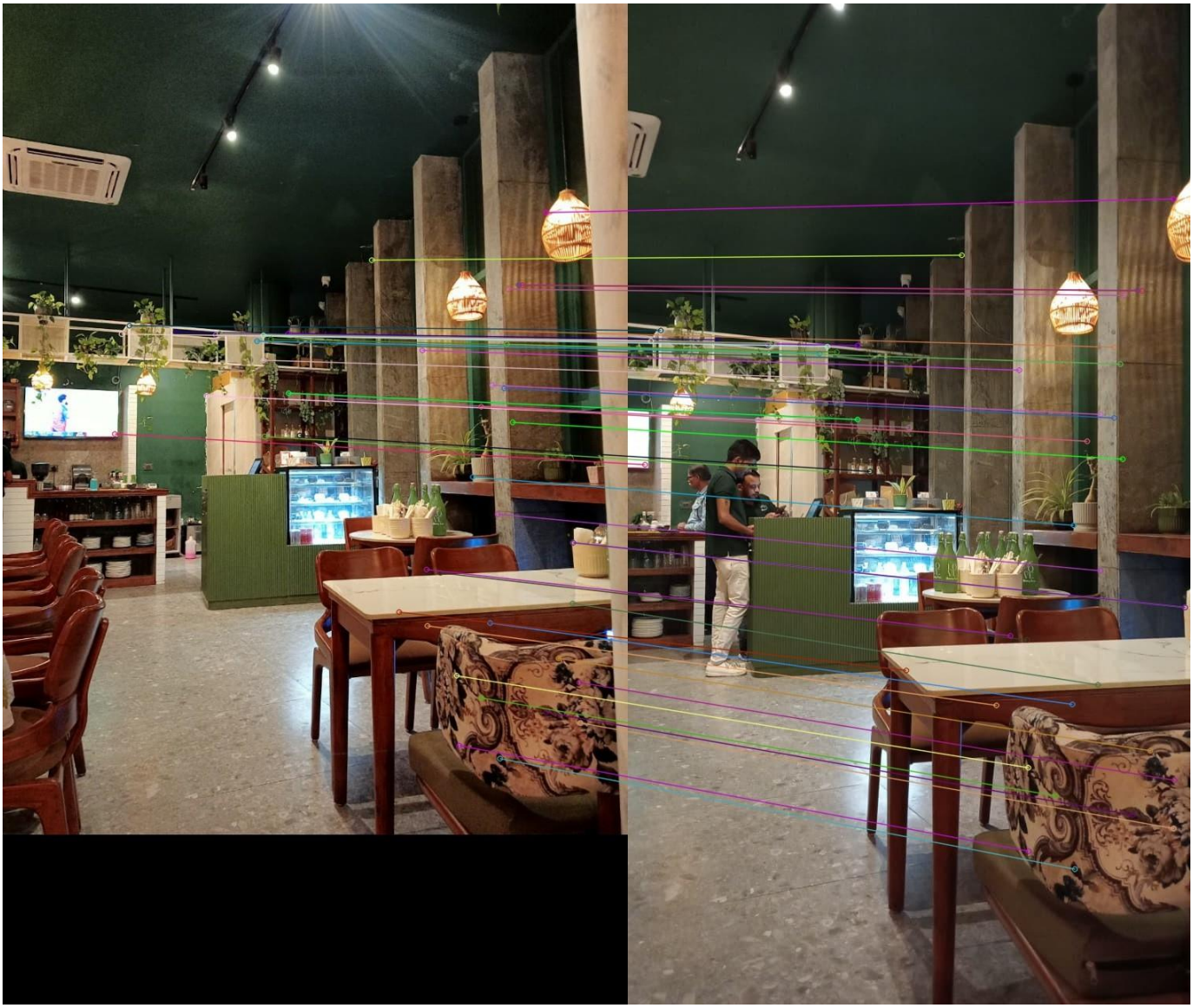


Figure 12 – Photo 1 vs photo 2.

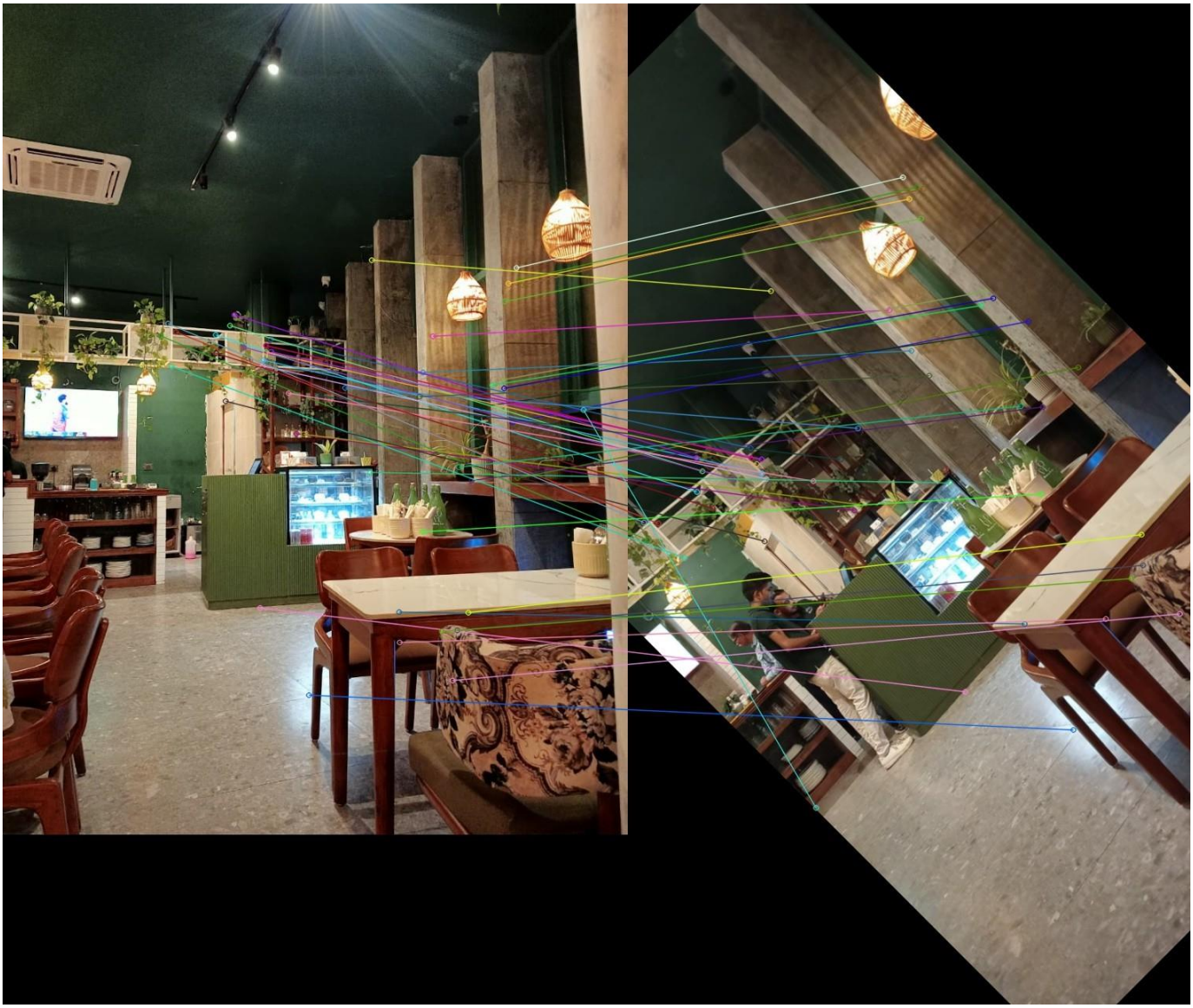


Figure 13 – Photo 1 vs photo 2 rotated 45°.

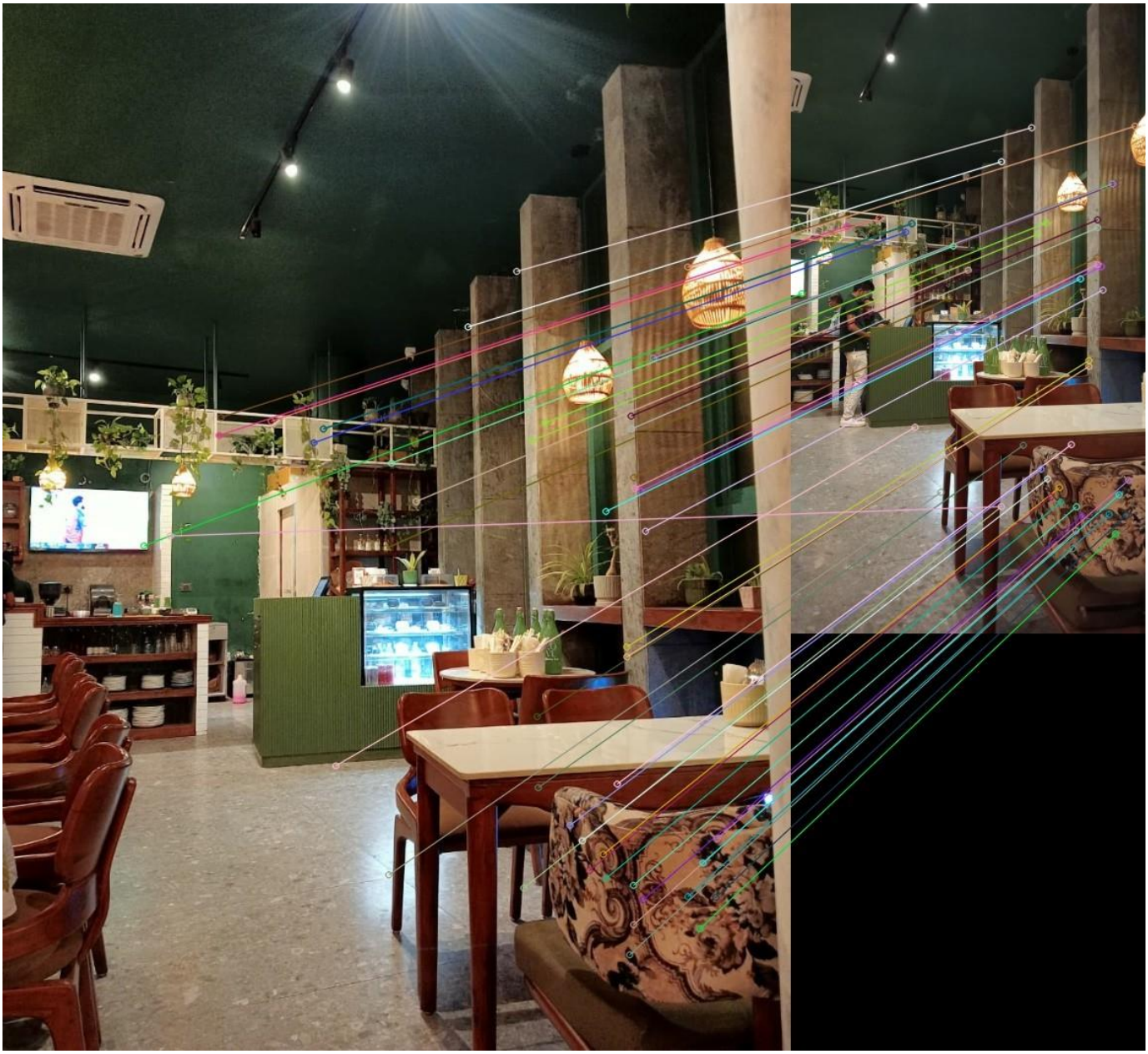


Figure 14 – Photo 1 vs photo 2 resized to 50%.

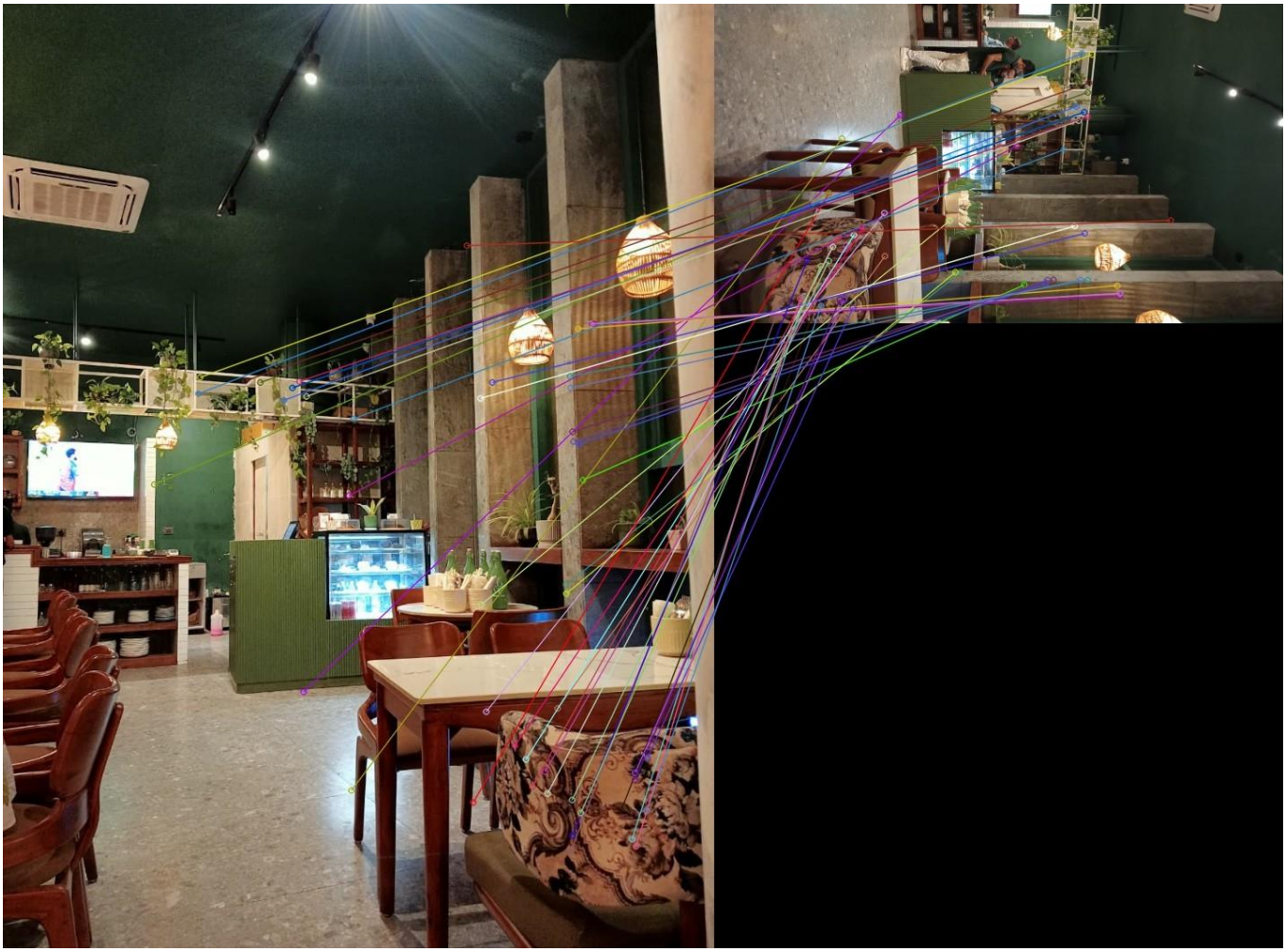


Figure 15 – Photo 1 vs photo 2 rotated 90° and resized to 50%.

Experiment	Keypoints (photo 1)	Keypoints (photo 2)	Good matches	RANSAC inliers	Inlier %
Photo 1 vs Photo 2 (original)	4053	3553	1185	604	51.0
Photo 1 vs Photo 2 rotated 45°	4053	2826	780	581	74.5
Photo 1 vs Photo 2 resized 50%	4053	1203	637	447	70.2
Photo 1 vs Photo 2 rotated 90° + 50%	4053	1194	629	463	73.6

Observation: The SIFT + Lowe ratio test + RANSAC pipeline found 604 inliers from 1,185 good matches for the original pair (51.0%). In the transformed-photo experiments, inlier proportions ranged from 70.2% to 74.5%, with 447–581 inliers. The smaller transformed images produced fewer keypoints and good matches, but still provided substantial geometrically consistent matches. These results show that matching remained possible under the tested rotation and scale changes; they do not prove invariance for all images or viewpoints. HOG describes gradients on a fixed grid, so it is more sensitive to image alignment than local SIFT descriptors.

---

6. Task 3 — Object recognition: Viola–Jones vs YOLO (Unit 5)

Viola–Jones uses OpenCV's pretrained `haarcascade_frontalface_default.xml` (scaleFactor 1.1, minNeighbors 5). YOLO uses pretrained `yolov8n.pt` (COCO, 80 classes, confidence ≥ 0.25). The supplied recognition image contains several people. The video frames show a brief sequence from the supplied portrait-format video.

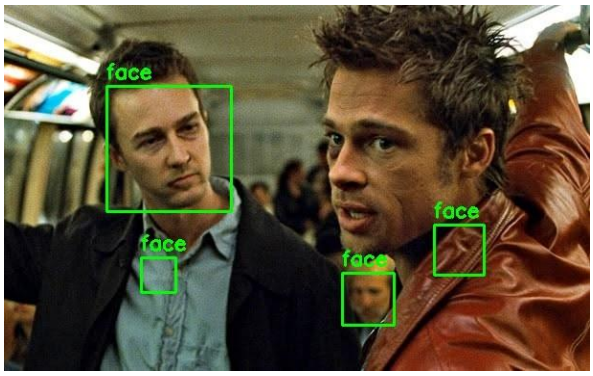
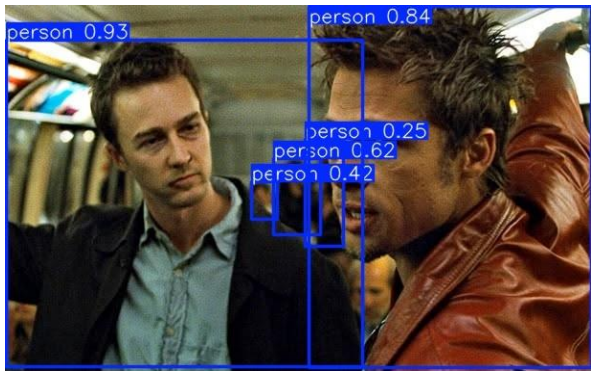
Haar cascade	YOLO
	
	
	
	

Figure 16 – Detections (left: Haar faces, right: YOLO boxes with class and confidence).

Image	Haar: faces found	Haar time (ms)	YOLO: objects (confidence)	YOLO time (ms)
Photo	4	175.1	5 detections: person (0.93, 0.84, 0.62, 0.42, 0.25)	46.4
Frame 25%	0	136.9	person (0.56)	72.1
Frame 50%	0	143.9	car (0.67), motorcycle (0.65), person (0.50)	47.2
Frame 75%	0	139.3	none	49.0

6.1 Failure cases

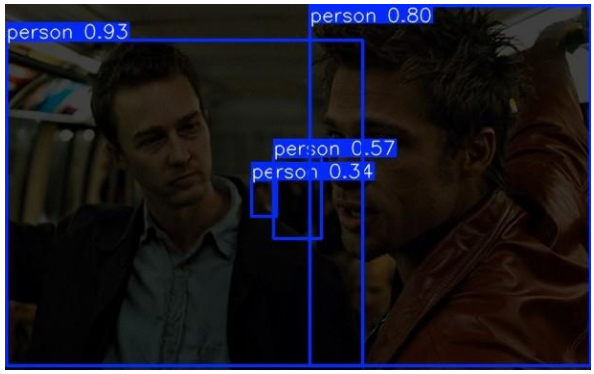
Haar	YOLO
	
	
	

Figure 17 – Rotated, dark and small versions of the test image.

Test	Haar: faces found	YOLO: objects (confidence)
Rotated 90°	0	dog (0.39), person (0.29)
Very dark	1	4 person detections (0.93, 0.80, 0.57, 0.34)
Very small (200 px)	1	4 person detections (0.92, 0.64, 0.61, 0.35)

Failure cases observed: The face cascade returned four face detections on the supplied photo, while YOLO returned five person detections. On the sampled video frames, Haar returned zero detections; YOLO found a person at 25%, a car, motorcycle and person at 50%, and no objects at 75%. The transformed-image tests also show sensitivity to rotation: Haar found no face after a 90° rotation. YOLO's rotated-image predictions included a low-confidence dog and person; these may be misclassifications and should be checked visually. Haar and YOLO counts are model outputs and may include false or duplicate detections, so they are not ground-truth counts.

6.2 Comparison — Task 3

	Viola–Jones (Haar)	YOLO (v8n)
Classes	frontal face only	80 COCO classes

	Viola–Jones (Haar)	YOLO (v8n)
Output	box only	box + class + confidence
Speed on this run	136.9–175.1 ms per image	46.4–72.1 ms per image
Small image (200 px)	found 1 face	4 person detections (0.92, 0.64, 0.61, 0.35)
Possible false detections	verify the four photo detections visually	dog (0.39) in the rotated test; other detections need visual verification

On this run, YOLO inference was faster than Haar for each tested input. The timings are machine- and run-dependent and include only the notebook's measured detection call.

---

7. Task 4 — Video motion estimation (Unit 5)

The supplied video has 297 frames at approximately 60 fps (about 4.95 s), with source dimensions 720 × 1280 pixels. Frames are resized to 480 × 270 for processing.



Frame T	*T + 1*	*T + 2*	*T + 3*

Figure 18 – Four consecutive frames.

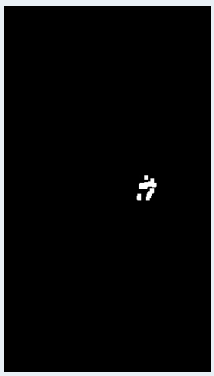


Figure 19 – Frame differencing mask (threshold 25)



Figure 20 – GMM (MOG2) foreground mask

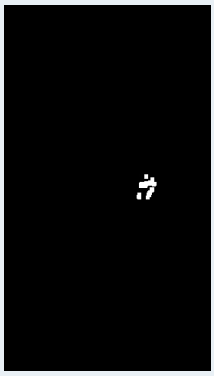
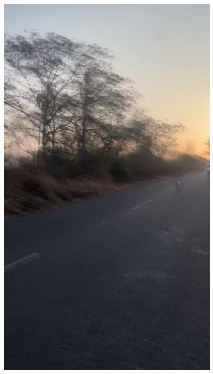
	
	
Figure 21 – Background learned by MOG2	*Figure 22 – Lucas–Kanade motion vectors (drawn 5× longer)*



Figure 23 – Paths of points tracked over 20 frames.

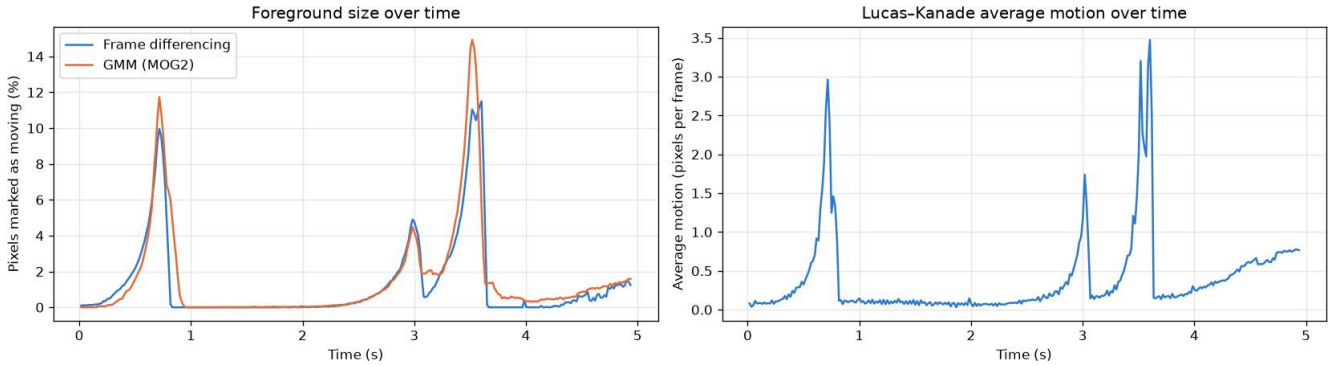


Figure 24 – Foreground size (frame differencing vs GMM) and average optical-flow motion over the whole video.

Method	Output	Avg time per frame (ms)	Average result
Frame differencing	binary mask	0.52	1.30% pixels moving
GMM background subtraction	binary mask	5.00	1.54% pixels moving
Lucas–Kanade optical flow	motion vectors	4.46	0.39 px motion per point

For the studied frame pair, Lucas–Kanade tracked 300 points; 5 moved more than 1 pixel. The reported average motion was 0.11 px, with a maximum of 1.94 px.

Observation: Frame differencing was the fastest method in this run (0.52 ms per frame), followed by Lucas–Kanade (4.46 ms) and GMM (5.00 ms). The two foreground methods marked similar proportions of pixels as moving (1.30% for frame differencing and 1.54% for GMM). The video is short, so these measurements characterize only this clip and run. Lucas–Kanade tracked 300 points in the illustrated frame pair, with only 5 moving by more than 1 pixel; this is consistent with the low overall motion measured in that pair.

7.1 Comparison — Task 4

	Frame differencing	GMM (MOG2)	Lucas–Kanade
Output	mask of changed pixels	mask of foreground	motion vectors
Object shape	outline only (hollow)	full silhouette	sparse points
Learning needed	no	yes (background model adapts over time)	no
Object stops	disappears	fades slowly	vectors ≈ 0
Speed (this run)	0.52 ms	5.00 ms	4.46 ms
Result on supplied video	1.30% pixels moving on average	1.54% pixels moving on average	0.39 px average motion per point over the video

8. Conclusion

This project applied image restoration, segmentation, feature matching, object recognition and motion estimation to the supplied images and video. Gaussian filtering gave the better result on Gaussian noise (27.49 dB, SSIM 0.704), while median filtering was better on salt-and-pepper noise (28.35 dB, SSIM 0.852). Otsu selected a threshold of 100, and Canny produced the fewest edges after blurring with thresholds (100, 200) (3.96%, compared with 19.36% on the noisy image). Watershed produced 25 regions from the supplied textured flat-lay image; these regions are not a verified physical-object count. SIFT matching produced 604 RANSAC inliers for the original photo pair and 447–581 inliers in the rotation/scale experiments, while HOG feature lengths differed between the two photos because their dimensions differ.

On the supplied recognition image, Haar returned four face detections and YOLO returned five person detections; these are predictions that require visual checking, not ground truth. YOLO was faster than Haar for the measured inputs in this run. On the short video, frame differencing was fastest (0.52 ms per frame); frame differencing and GMM marked 1.30% and 1.54% of pixels as moving on average. Lucas–Kanade provided sparse motion vectors, with low measured motion in the selected frame pair. These results depend on the input data and detector settings, and should be interpreted alongside the generated visual outputs.

---

9. References

1. OpenCV documentation – https://docs.opencv.org/4.x/
2. scikit-image documentation – https://scikit-image.org/docs/stable/
3. Ultralytics YOLO documentation – https://docs.ultralytics.com/
4. N. Otsu, “A threshold selection method from gray-level histograms”, 1979.
5. J. Canny, “A computational approach to edge detection”, 1986.
6. Z. Wang et al., “Image quality assessment: from error visibility to structural similarity” (SSIM), 2004.
7. N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection”, 2005.
8. D. Lowe, “Distinctive image features from scale-invariant keypoints” (SIFT), 2004.
9. P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features”, 2001.
10. J. Redmon et al., “You Only Look Once: unified, real-time object detection”, 2016.
11. Z. Zivkovic, “Improved adaptive Gaussian mixture model for background subtraction” (MOG2), 2004.
12. B. Lucas and T. Kanade, “An iterative image registration technique with an application to stereo vision”, 1981.